11.1 Authorization in a microservices world

For ACME, moving to microservices changed how authorization was handled. The original Customer Collaboration platform was a single monolith, with all authorization checks hidden neatly inside one codebase. But when teams split that system into separate services for documents, search, and notifications, each new service boundary became an exposed API surface. HR and Finance followed this approach, creating APIs so their data could be shared across departments. What once felt like a single, well-guarded front door became a neighborhood of smaller doors, each with its own lock and key.

The transition was exciting for engineers like Alice, who enjoyed the speed and autonomy of creating new services. But it also revealed some flaws. Some APIs had strict authorization rules, others had only basic role checks, and a few assumed their callers were already trusted.

Bob, ACME's solutions architect, could see the bigger picture. He mapped the growing service graph and realized that a single request—like Carol, a manager, asking for a project report—could touch a dozen APIs. If each service were responsible for its own checks, maintaining consistency would be impossible. "We had the classic microservices problem," Bob said. "Lots of flexibility, but authorization that was fragmented and brittle."

The identity team recognized the need for a systematic approach. They examined architectural patterns for deploying policy enforcement in a distributed system. Three options stood out as practical: using the API gateway as a centralized enforcement point, implementing enforcement within a service mesh, and deploying per-service proxies in front of critical APIs. Each option had its trade-offs, and ACME ultimately chose a combination.